

Python for AI & Machine Learning

Class 24 – Data Aggregation and Grouping with Pandas

Course: Python for AI & Machine Learning

Module: Data Analysis with Python

Duration: 2 Hours (Theory + Practical)

Learning Objectives

By the end of this session, students will be able to:

- Understand Data Aggregation.
 - Group data using `groupby()`.
 - Apply aggregate functions.
 - Sort grouped data.
 - Perform multi-level grouping.
 - Create Pivot Tables.
 - Analyze datasets for AI and Machine Learning.
-

Revision

In the previous class, we learned:

- Data Cleaning
- Missing Values
 - `isnull()`
 - `dropna()`
 - `fillna()`
- Duplicate Records
- Renaming Columns
- Changing Data Types
- String Operations
 - `apply()`

Today we will learn how to summarize and analyze large datasets using **Grouping and Aggregation**.

What is Data Aggregation?

Data Aggregation is the process of combining multiple rows of data to generate meaningful summaries.

Examples:

- Average salary department-wise
- Total sales month-wise
- Student average marks class-wise
- Product sales category-wise

Aggregation helps in making better business decisions.

What is `groupby()` ?

The `groupby()` function groups rows that have the same values in one or more columns.

Syntax

```
df.groupby("column_name")
```

Sample Dataset

```
import pandas as pd

students = {
    "Name": ["Rahul", "Sneha", "Aman", "Priya", "Kiran"],
    "Course": ["Python", "Python", "AI", "AI", "Python"],
    "Marks": [85, 92, 78, 95, 88]
}

df = pd.DataFrame(students)

print(df)
```

Grouping Data

```
group = df.groupby("Course")  
  
print(group)
```

This creates grouped data but does not display summaries yet.

Average Marks by Course

```
print(df.groupby("Course")["Marks"].mean())
```

Output

Course	
AI	86.5
Python	88.3

Total Marks by Course

```
print(df.groupby("Course")["Marks"].sum())
```

Maximum Marks

```
print(df.groupby("Course")["Marks"].max())
```

Minimum Marks

```
print(df.groupby("Course")["Marks"].min())
```

Count Students

```
print(df.groupby("Course")["Name"].count())
```

Multiple Aggregations

```
print(
    df.groupby("Course")["Marks"].agg(
        ["mean", "max", "min", "sum", "count"]
    )
)
```

Output

	mean	max	min	sum	count
Course					
AI	86.5	95	78	173	2
Python	88.3	92	85	265	3

Grouping by Multiple Columns

```
employees = {
    "Department": ["HR", "HR", "IT", "IT", "Sales"],
    "City": ["Hyderabad", "Delhi", "Hyderabad", "Delhi", "Delhi"],
    "Salary": [35000, 42000, 60000, 65000, 45000]
}

df = pd.DataFrame(employees)

print(
    df.groupby(["Department", "City"])["Salary"].mean()
)
```

Sorting Grouped Data

```
result = df.groupby("Department")["Salary"].mean()

print(result.sort_values(ascending=False))
```

Resetting Index

```
result = df.groupby("Course")["Marks"].mean()

print(result.reset_index())
```

Pivot Table

A Pivot Table summarizes data in a table format.

```
import pandas as pd

sales = {
    "Month": ["Jan", "Jan", "Feb", "Feb"],
    "Product": ["Laptop", "Mobile", "Laptop", "Mobile"],
    "Sales": [50000, 30000, 60000, 35000]
}

df = pd.DataFrame(sales)

pivot = pd.pivot_table(
    df,
    values="Sales",
    index="Month",
    columns="Product",
    aggfunc="sum"
)

print(pivot)
```

Output

```
Product  Laptop  Mobile
Month
```

Feb	60000	35000
Jan	50000	30000

Value Counts

Count occurrences of values.

```
print(df["Month"].value_counts())
```

Unique Values

```
print(df["Product"].unique())
```

Number of Unique Values

```
print(df["Product"].nunique())
```

AI Example 1 – Student Performance Analysis

```
import pandas as pd

students = {
    "Course": ["AI", "AI", "Python", "Python"],
    "Marks": [88, 91, 75, 84]
}

df = pd.DataFrame(students)

print(df.groupby("Course")["Marks"].mean())
```

AI Example 2 – Employee Salary Report

```
employees = {  
    "Department": ["HR", "HR", "IT", "Sales"],  
    "Salary": [40000, 45000, 70000, 50000]  
}  
  
df = pd.DataFrame(employees)  
  
print(df.groupby("Department")["Salary"].sum())
```

AI Example 3 – Sales Dashboard

```
sales = {  
    "City": ["Hyderabad", "Delhi", "Hyderabad", "Mumbai"],  
    "Revenue": [250000, 180000, 300000, 220000]  
}  
  
df = pd.DataFrame(sales)  
  
print(df.groupby("City")["Revenue"].sum())
```

AI Example 4 – Hospital Analytics

```
patients = {  
    "Department": ["Cardiology", "Orthopedics", "Cardiology"],  
    "Patients": [120, 90, 150]  
}  
  
df = pd.DataFrame(patients)  
  
print(df.groupby("Department")["Patients"].sum())
```

Best Practices

- Clean data before grouping.
- Use meaningful column names.
- Choose appropriate aggregation functions.
- Use Pivot Tables for business reports.

- Reset the index after grouping if needed.
-

Practical Exercise 1

Create a DataFrame containing:

- Student Name
- Course
- Marks

Calculate:

- Average Marks by Course
 - Highest Marks by Course
-

Practical Exercise 2

Create an Employee DataFrame.

Calculate:

- Department-wise Salary
 - Maximum Salary
 - Minimum Salary
-

Practical Exercise 3

Create a Sales DataFrame.

Generate a Pivot Table showing monthly sales by product.

Practical Exercise 4

Display:

- Unique Courses
 - Number of Unique Courses
 - Count of Students in each Course
-

Practical Exercise 5

Group by two columns:

- Department
- City

Calculate average salary.

Assignment

Student Performance Analytics Dashboard

Create a program that:

- Reads a student CSV file.
 - Groups students by course.
 - Calculates:
 - Average Marks
 - Highest Marks
 - Lowest Marks
 - Total Students
 - Generates a Pivot Table.
 - Displays unique courses.
 - Saves the summary as **student_summary.csv**.
-

Interview Questions

1. What is Data Aggregation?
 2. What is the purpose of `groupby()`?
 3. What are aggregate functions?
 4. What is a Pivot Table?
 5. Explain `value_counts()`.
 6. What is the difference between `unique()` and `nunique()`?
 7. How do you group data using multiple columns?
 8. Why is aggregation important in Data Analysis?
 9. How is `groupby()` used in Machine Learning projects?
 10. Explain a real-world application of Pivot Tables.
-

Summary

Today we learned:

- Data Aggregation
- `groupby()`
- Aggregate Functions
- Multiple Aggregations
- Multi-level Grouping
- Pivot Tables
- `value_counts()`
- `unique()`
- `nunique()`
- AI & Machine Learning Applications

Grouping and aggregation are essential techniques for summarizing large datasets. They help data analysts and Machine Learning engineers discover patterns, trends, and insights before building predictive models.

Next Class Preview

Class 25 – Data Visualization with Matplotlib

Topics to be covered:

- Introduction to Matplotlib
- Installing Matplotlib
- Line Charts
- Bar Charts
- Pie Charts
- Histograms
- Scatter Plots
- Customizing Graphs
- Saving Charts
- Data Visualization for AI & Machine Learning